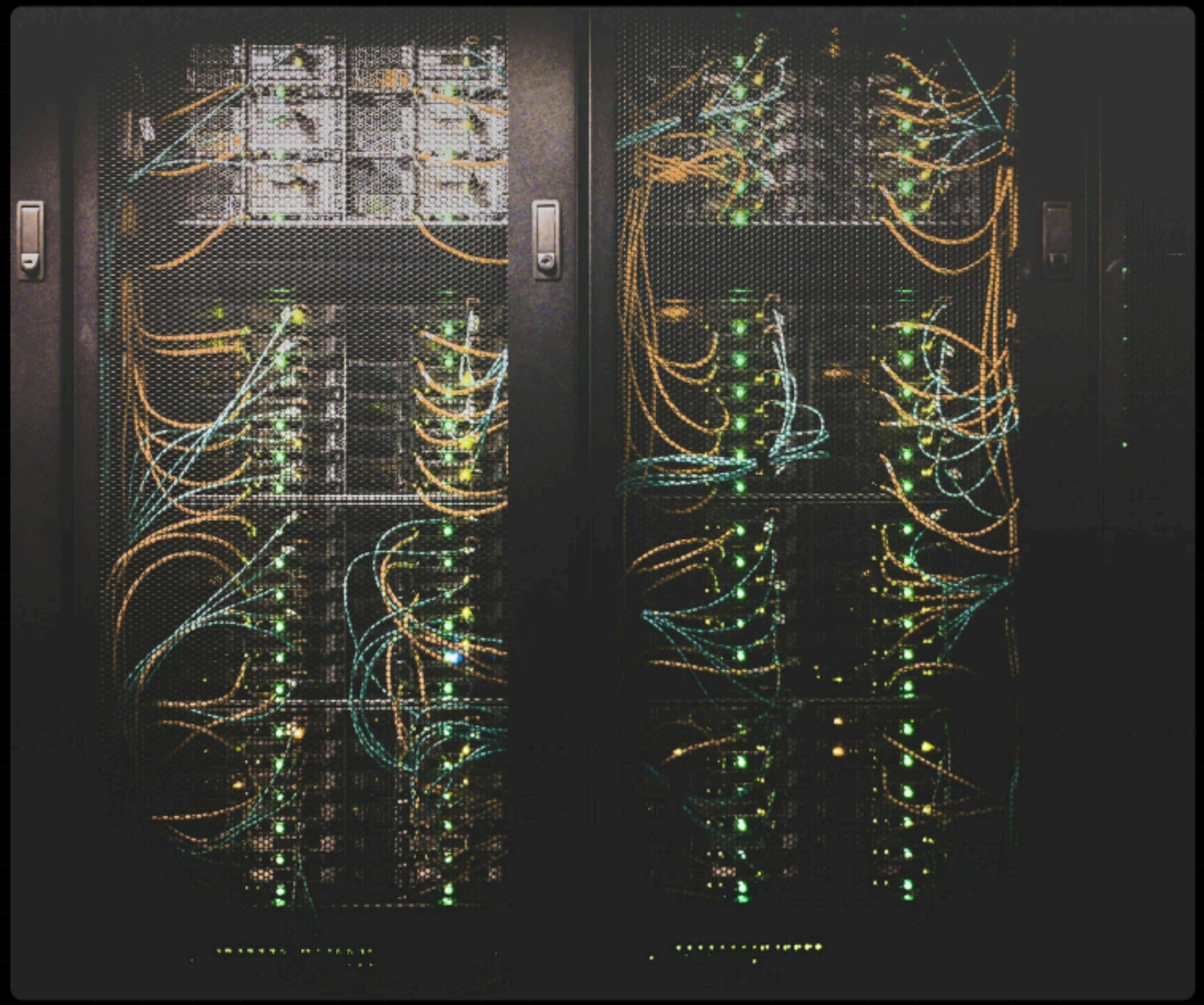
Advanced RAG Evaluation and Optimization Techniques

A scientific approach to measuring and maximizing the performance of production-grade Retrieval-Augmented Generation systems.
Moving beyond empirical debugging to quantitative performance measurement and optimization.



Recap - What We've Built So Far

Implemented basic character-based chunking and standard embedding models.

Established baseline RAG pipelines using standard vector stores and LLM providers.

Integrated initial user query interfaces for document interaction.

The RAG Reality Check

Transitioning from empirical 'whack-a-mole' debugging to data-driven engineering. Relying on vibes or single-query tests is insufficient for production. We must move to **quantitative measurement** to ensure reliability.

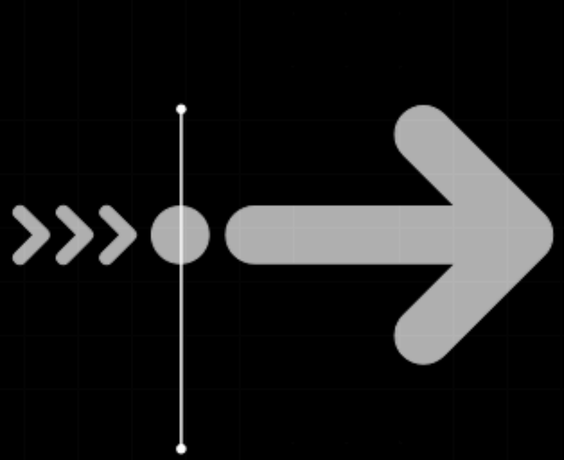
Why Evaluations Matter

Without measurement, optimization is just guessing; evaluations provide the feedback loop required to justify architectural changes and model selections.

The Three Steps of RAG Evaluation

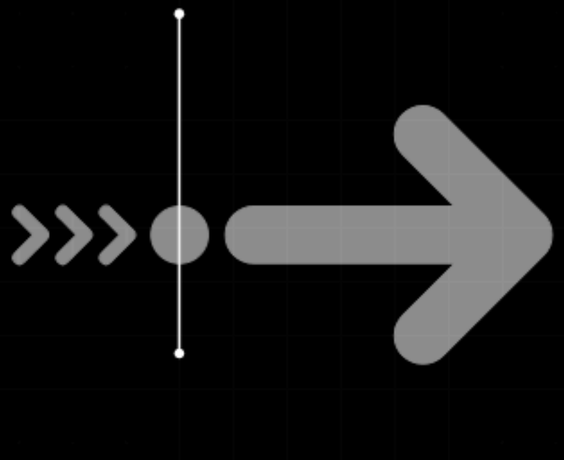
Build Golden Dataset

Curate test questions, reference answers, and keywords derived from real user data.



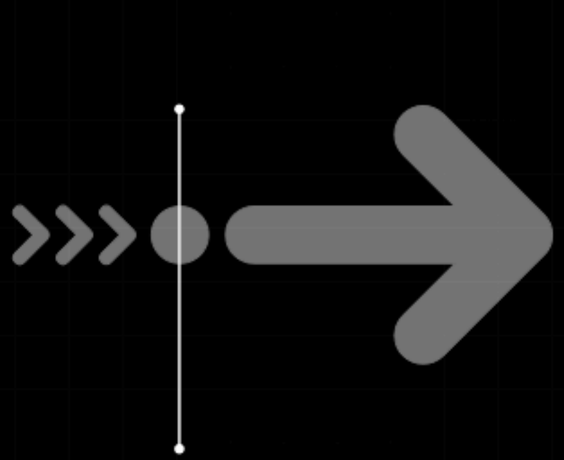
Measure Retrieval

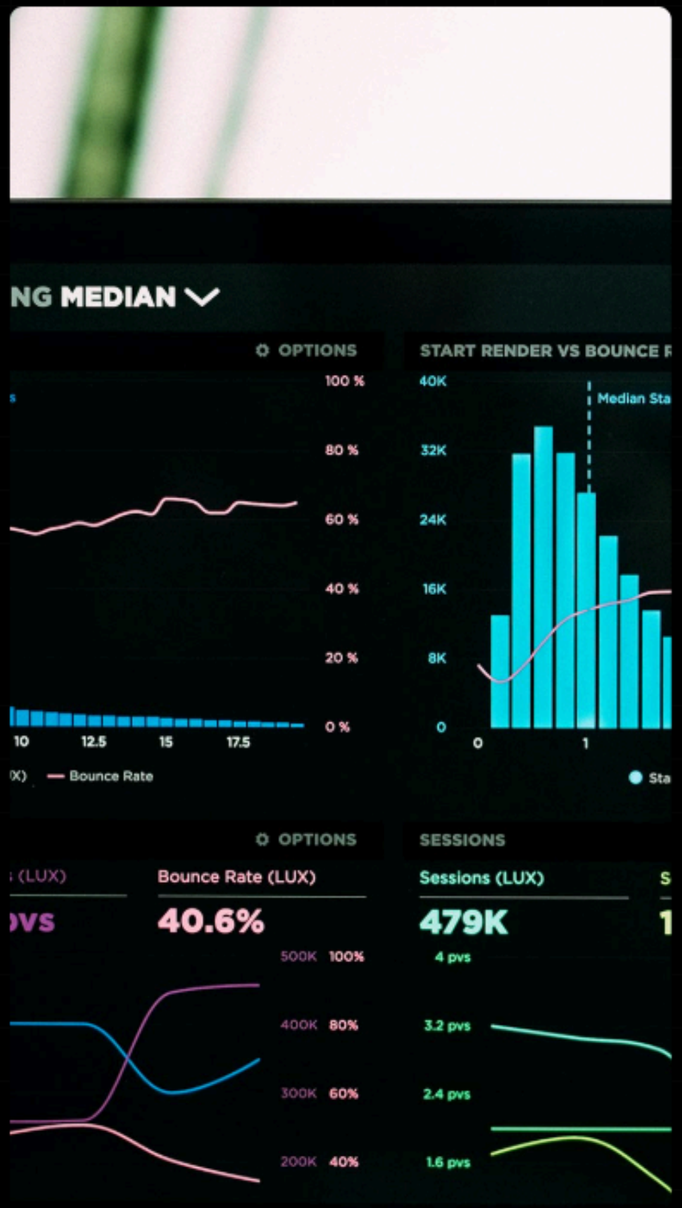
Evaluate the vector search performance using metrics like MRR, NDCG, and Recall.



Measure Answers

Assess the final generated output quality using the LLM-as-a-Judge methodology.





The Core Retrieval Metrics

These KPIs provide a quantitative look at retrieval quality and information positioning.

Mean Reciprocal Rank

MRR

Measures the rank of the first relevant chunk. Calculation: 1/rank (1st = 1.0, 2nd = 0.5).

NDCG Score

NDCG

Normalized Discounted Cumulative Gain penalizes relevant content appearing late; perfect score is 1.0.

System Recall

RECALL

Percentage of tests where the top-k chunks contain hits. Critical for ensuring no info is missed.

Precision vs. Recall Trade-offs

Technical definitions for engineering teams building production pipelines.

Recall represents the percentage of cases where the top-k chunks contain hits; this is our high-priority metric.

Precision measures the percentage of retrieved chunks that are relevant. We accept lower precision for higher recall.

High Recall ensures info is not missed, which is critical for **RAG system reliability**.

LLM-as-a-Judge

Accuracy
Score factual correctness of the generated answer against the gold reference using a high-capacity model (e.g., GPT-4).

Completeness
Evaluate whether the answer addresses all parts of the user query and includes necessary details or missing elements.

Relevance
Assess how directly the answer matches the user's intent and context, filtering out tangential or irrelevant content.

Experimental Results - Baseline

0.73

Baseline MRR

Initial Mean Reciprocal Rank using standard retrieval settings.

0.739

Baseline NDCG

Initial Normalized Discounted Cumulative Gain performance.

83.8 %

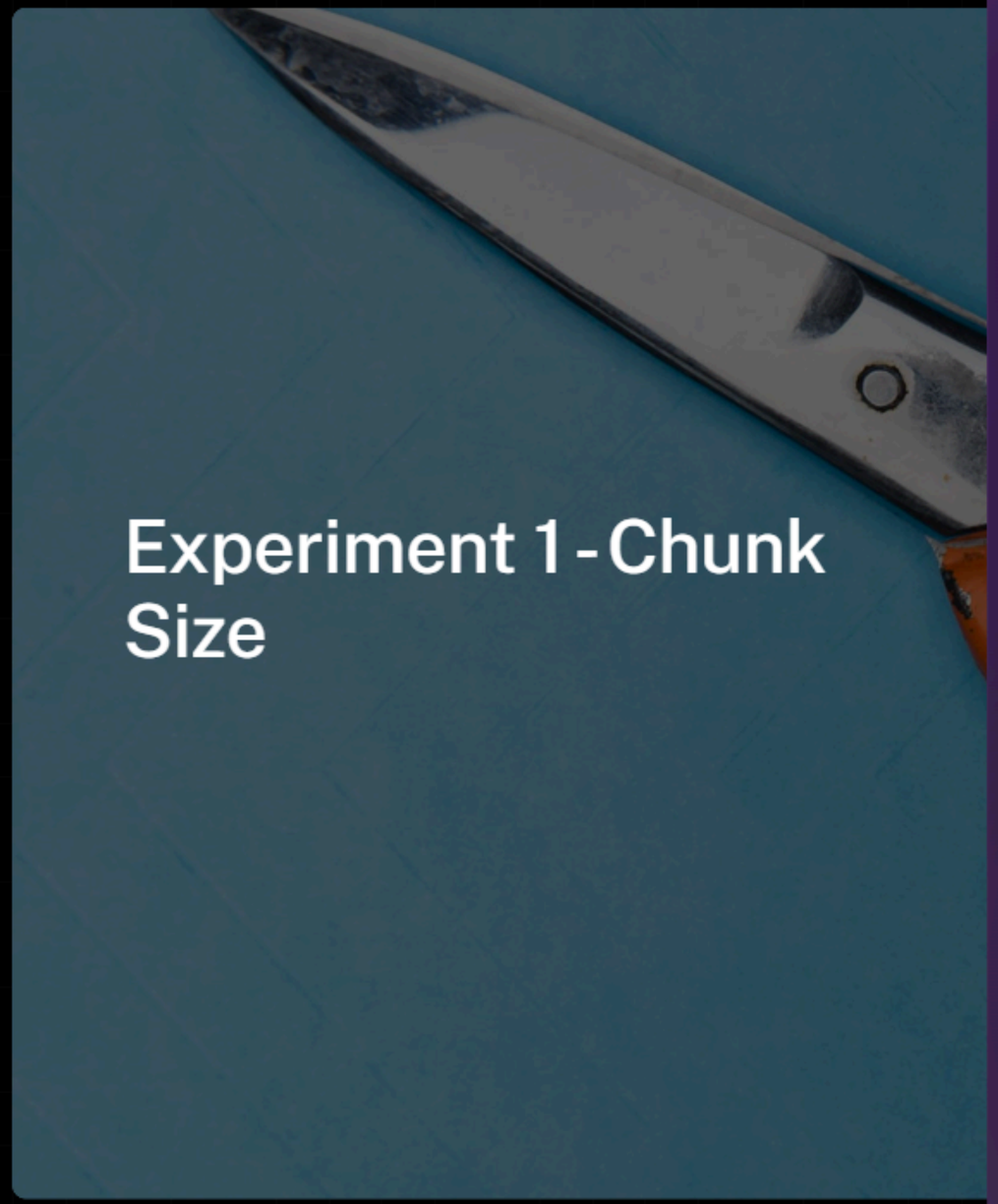
Keyword Coverage

Percentage of golden keywords found in retrieved chunks.

3.99

Answer Accuracy

Baseline LLM-as-a-Judge score out of 5.0.



Experiment 1 - Chunk Size

0.76

Optimized MRR

Retrieval performance achieved with a 500-character chunk size.

500

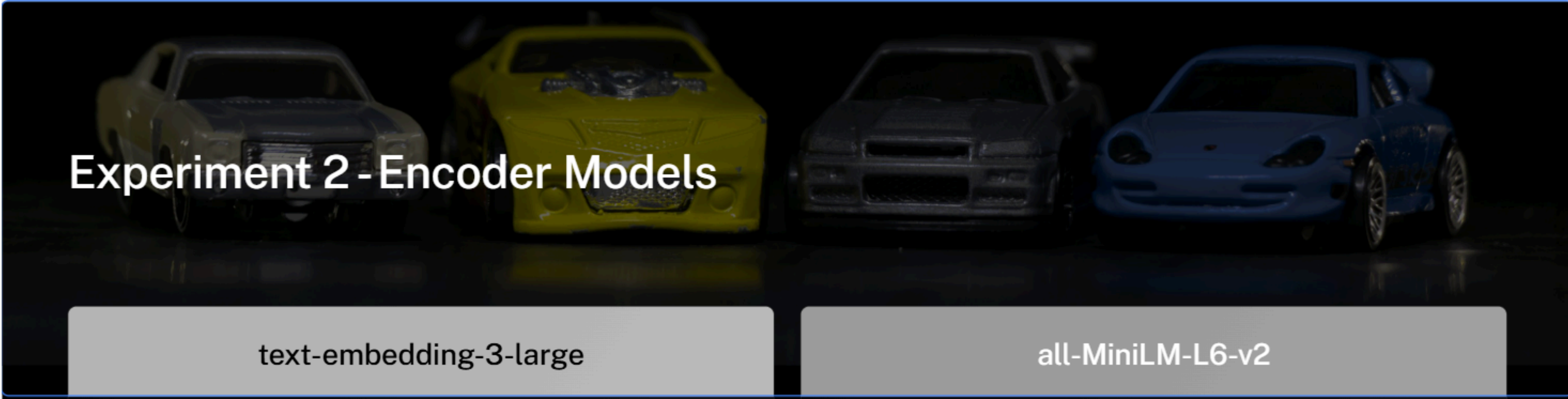
Best Performer

The character count that yielded the highest retrieval metrics.

1,000+

Larger Chunks

Both 1000 and 1667 character chunks showed inferior MRR results.



Experiment 2 - Encoder Models

text-embedding-3-large	all-MiniLM-L6-v2
0.79	**Lower**
MRR achieved using OpenAI's latest large embedding model.	Standard open-source model significantly underperformed in the tests.

The Zoo of Advanced RAG Techniques

1. Semantic Chunking & Encoders
2. Prompts & Pre-processing
3. Query Rewriting & Expansion
4. Reranking Logic
5. Hierarchical & Graph RAG
6. Agentic Implementation

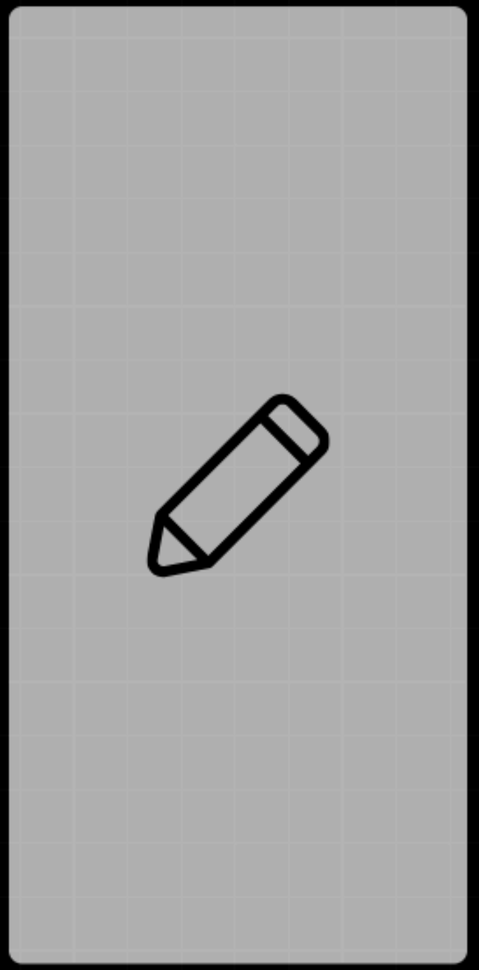
Technique 1 & 2 - Chunking & Encoders

Semantic Chunking uses an LLM to split text into meaningful sections rather than arbitrary character counts.

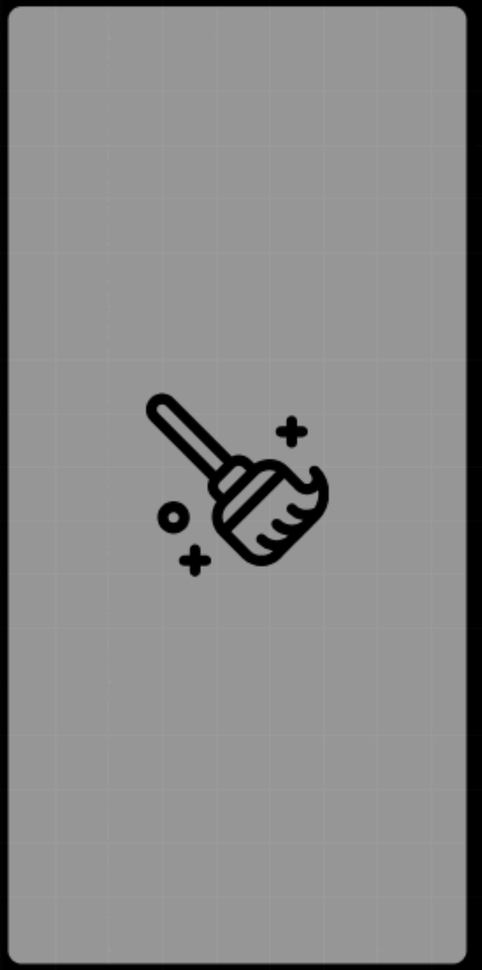
Aligning chunks with **semantic meaning** significantly improves retrieval accuracy over fixed-length windows.

Choosing high-performing models like **text-embedding-3-large** provides a superior baseline for vector space.

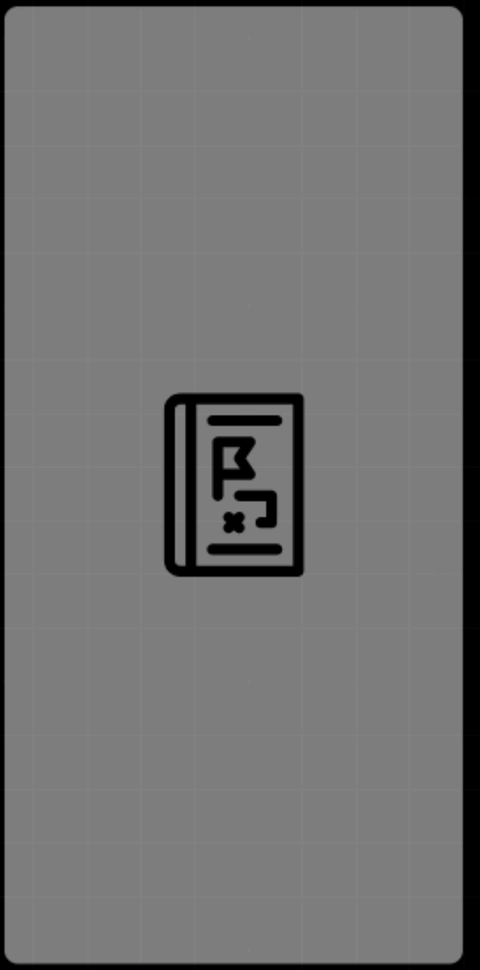
Technique 3 & 4 - Prompts & Pre-processing



Prompt Engineering remains a high-impact, low-effort optimization that is frequently overlooked by engineers.



Implement **Document Pre-processing** to clean noise, remove duplicates, and normalize text before indexing.



Use structured instructions to guide the retriever on how to prioritize specific data attributes.

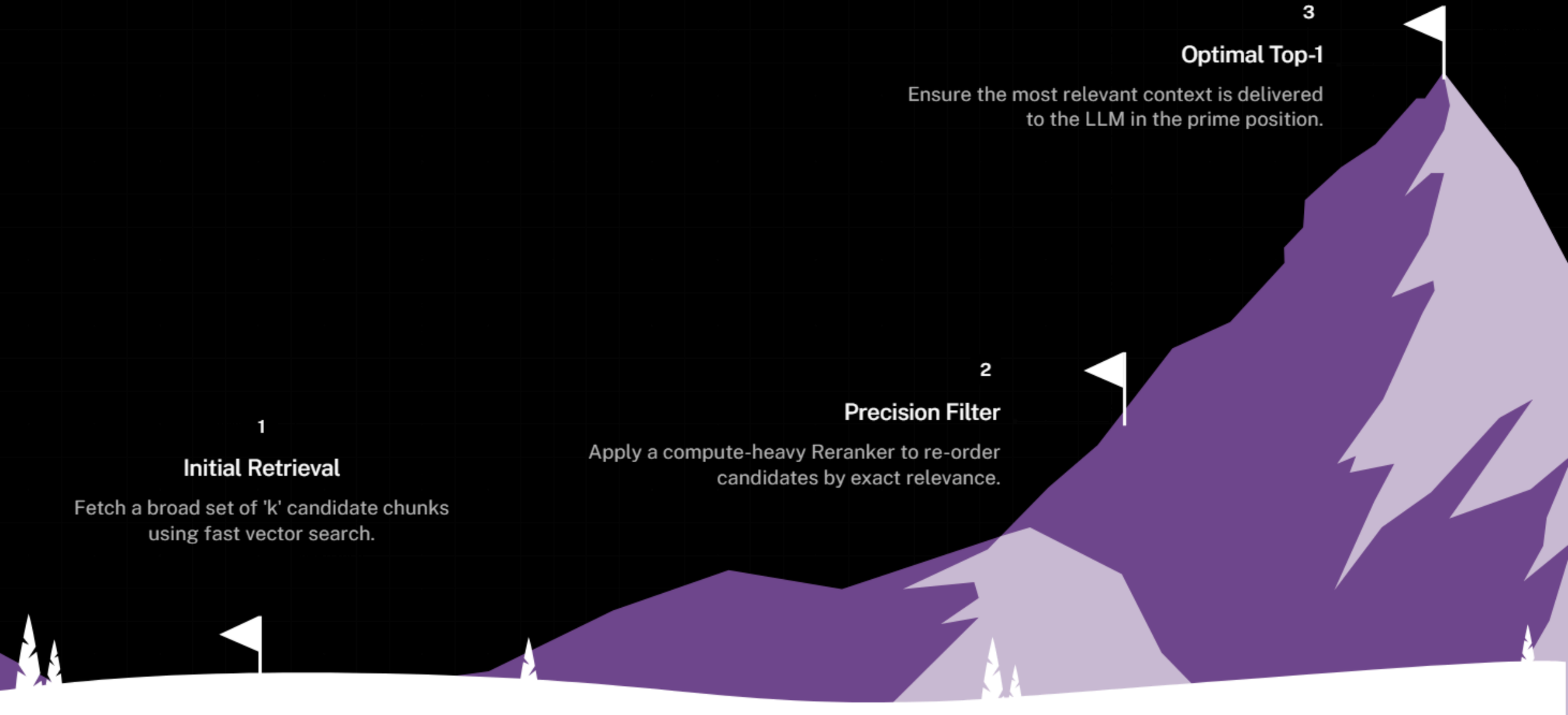
Technique 5 & 6 - Query Rewriting & Expansion

Query Expansion Involves Generating Variations Of A Query And Merging The Results Of Multiple Searches.

This Increases **Recall** By Covering Multiple Semantic Angles And Synonyms That The User Might Have Missed.

Query Rewriting Clarifies Ambiguous User Questions Into Specific Search-Friendly Prompts.

Technique 7 - Reranking



1

Initial Retrieval

Fetch a broad set of 'k' candidate chunks using fast vector search.

2

Precision Filter

Apply a compute-heavy Reranker to re-order candidates by exact relevance.

3

Optimal Top-1

Ensure the most relevant context is delivered to the LLM in the prime position.

Technique 8 - Hierarchical RAG

Index Summaries Of Documents Alongside Raw Chunks To Provide Multi-Level Context Access.

Navigate From High-Level Summaries Down To Granular Details For Complex Question Answering.

Optimizes Context Window Usage By Providing Only The Necessary Level Of Detail.

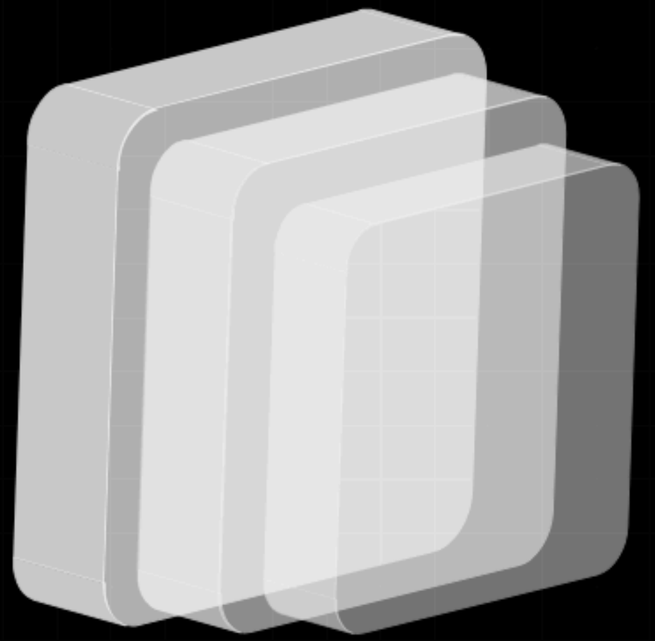
Technique 9 - Graph RAG

Relationship Retrieval

Uses metadata or graph databases like Neo4j to retrieve related chunks via hops.

Complex Reasoning

Enables the system to answer questions that require traversing multiple data points.



Connected Entities

Ideal for data where relationships between entities are as important as the text itself.

Technique 10 - Agentic RAG

Provides extreme flexibility but introduces **higher latency and cost** compared to fixed pipelines.

Agentic RAG handles unpredictable questions and multi-step reasoning autonomously.

The LLM decides which tools (**Vector, SQL, Web**) to use for specific retrieval tasks.

Is RAG Dead?

Addressing the myth that long-context LLMs eliminate the need for retrieval. Even with 1M+ context windows, RAG remains essential for **cost-efficiency**, low latency, and factual grounding in massive datasets.

Advanced Implementation - No LangChain

Build direct integrations to maintain full control over the retrieval and ranking logic.

Avoid 'black-box' overhead by implementing your own orchestration and evaluation loops.

Enables easier debugging and performance profiling of each specific step in the pipeline.

Semantic Chunking with LLMs

1. Identify logical break points in text based on topic changes rather than word counts.
2. Significantly improves the **relevance of retrieved content** by preserving contextual integrity.
3. Requires a fast, cheaper LLM like GPT-4o-mini to process documents at scale efficiently.

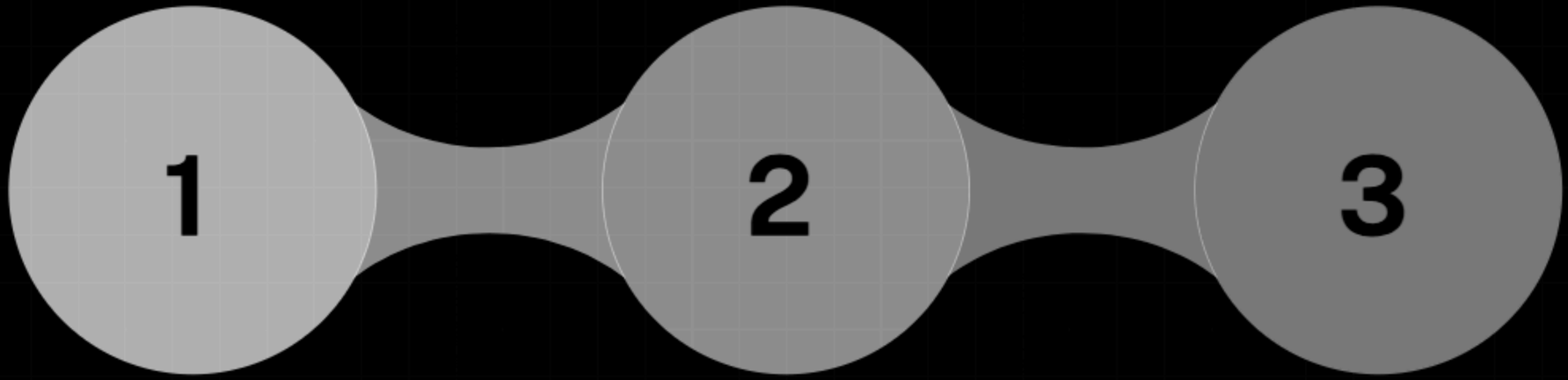
Structured Outputs for Consistency

Use **Pydantic** models to enforce strict schema adherence for LLM responses in your pipeline.

Ensures pipeline stability for LLM-based steps like **chunking** and **judging**.

Reduces parsing errors and allows for automated validation of retrieval metadata.

Reranking Implementation



Vector Search

Perform initial search to get the top 50 relevant chunks.

Reranker Scoring

Pass the query and 50 chunks to a Reranker model for cross-encoding.

Top-K Selection

Select the new top 5 chunks with the highest reranker scores for the prompt.

Query Expansion Strategy

Rewrite Query

Generate 3-5 different ways to ask the user's question.

Parallel Search

Execute vector search for all variations simultaneously.

Reciprocal Rank Fusion

Combine the results using RRF to find the most consistently high-ranking chunks.

Multiprocessing for Speed

Mitigating the high latency of LLM-based pre-processing and evaluation. Implement **multiprocessing** when performing batch tasks like semantic chunking or golden set generation to parallelize API calls and save hours of time.

Final Results - Advanced RAG

0.912

Final MRR

Top-tier retrieval performance after all optimizations.

0.903

Final NDCG

Near-perfect ranking distribution score.

4.62

Answer Accuracy

Final score out of 5.0 from LLM-as-a-Judge.

What Made the Difference?

1 Retrieval Improvements

- MRR increased by **25%** through chunk size and encoder tuning.
- Keyword coverage reached **96%** with query expansion.

2 Answer Quality Gains

- Overall accuracy improved by **16%** via reranking.
- Completeness was enhanced by larger context variety.

Best Practices Summary

Always build a Golden Test Set before attempting any architectural changes.

Prioritize **Reranking and Query Expansion** for the most consistent metric gains.

Use structured outputs and multiprocessing to ensure **production-grade reliability**.

Common Pitfalls

1

Focusing on **Precision over Recall**; remember that LLMs can't fix missing context.

2

Overlooking prompt engineering as a **high-impact** optimization lever.

3

Introducing complex agentic workflows when a **simple fixed pipeline** would suffice.

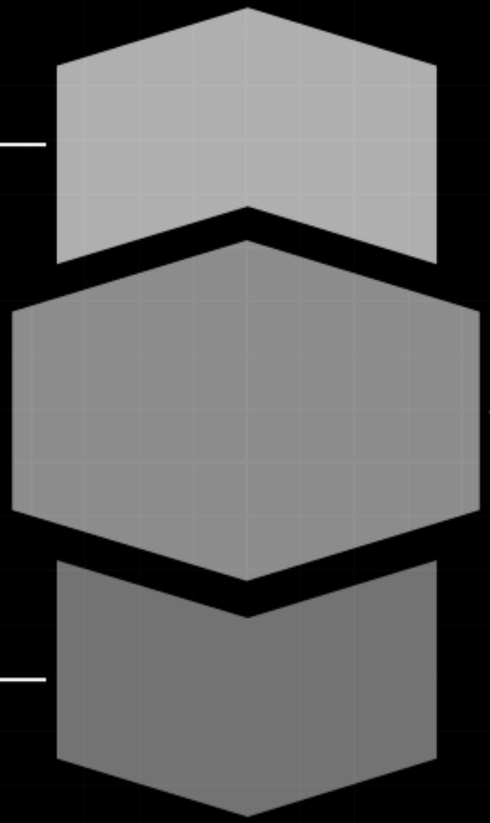
When Each Technique Helps

Technique	Core Benefit	Trade-off
Reranking	Boosts precision/relevance	Adds latency
Query Expansion	Maximizes recall	Increases API costs
Graph RAG	Relationship logic	Complex indexing
Agentic RAG	Extreme flexibility	Highest cost/time

The Evaluation Framework

Set Benchmark

Define the golden dataset of expected Q&A pairs.



Iterate Pipeline

Apply a single change (chunking, expansion, model).

Re-evaluate

Run metrics to quantify the improvement or regression.

Real-World Applications

Legal Discovery Tools leveraging high-recall Query Expansion across case law.

Financial Analysis Bots utilizing Semantic Chunking for quarterly reports.

Technical Support Agents using Graph RAG to traverse complex system manuals.

Thank You!